

Teach Module 15: Business Logic and Service Layer Design

This note teaches Module 15 using original explanations and examples. It uses the course outline only to identify the topic sequence.

Core Idea

In a Java/Spring application, the controller should not contain business rules. The controller handles HTTP, the repository handles data access, and the service layer owns the application's real behavior.

```
Client / UI
|
Controller -> handles HTTP requests/responses
|
Service     -> business decisions, workflows, validation beyond simple input shape
|
Repository  -> database operations
|
Database
```

A useful rule:

Controllers ask, services decide, repositories fetch/save.

Thin Controller Example

```
@RestController
@RequestMapping("/accounts")
public class AccountController {

    private final AccountService accountService;

    public AccountController(AccountService accountService) {
        this.accountService = accountService;
    }

    @PostMapping("/{id}/withdraw")
    public AccountResponse withdraw(
        @PathVariable Long id,
        @RequestBody WithdrawRequest request) {

        return accountService.withdraw(id, request.amount());
    }
}
```

The controller does not decide whether the account exists, whether the amount is valid, or whether the account has enough balance. That belongs in the service.

Service Example

```
@Service
public class AccountService {

    private final AccountRepository accountRepository;

    public AccountService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    @Transactional
    public AccountResponse withdraw(Long accountId, BigDecimal amount) {
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new InvalidAmountException("Withdrawal amount must be positive");
        }

        Account account = accountRepository.findById(accountId)
            .orElseThrow(() -> new AccountNotFoundException(accountId));

        if (account.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException(accountId);
        }

        account.withdraw(amount);

        return AccountResponse.from(account);
    }
}
```

The service:

- Finds needed data.
- Applies business rules.
- Updates domain state.
- Controls the transaction.
- Returns a useful result to the controller.

Why the Service Layer Matters

Without a service layer, controllers become crowded with business decisions:

```
@PostMapping("/{id}/withdraw")
public AccountResponse withdraw(@PathVariable Long id, @RequestBody WithdrawRequest request)
{
    Account account = accountRepository.findById(id).orElseThrow();

    if (request.amount().compareTo(BigDecimal.ZERO) <= 0) {
        throw new RuntimeException("Invalid amount");
    }
}
```

```

    if (account.getBalance().compareTo(request.amount()) < 0) {
        throw new RuntimeException("Insufficient funds");
    }

    account.setBalance(account.getBalance().subtract(request.amount()));
    accountRepository.save(account);

    return AccountResponse.from(account);
}

```

This works at first, but it creates problems:

- The controller becomes hard to read.
- Business rules are mixed with HTTP concerns.
- Testing requires web-layer setup.
- The same logic gets duplicated across endpoints.
- Transactions become unclear.
- Changing business rules becomes risky.

Service Boundaries

A service should represent a meaningful area of behavior.

Good service names:

```

OrderService
PaymentService
AccountService
EnrollmentService
InventoryService

```

Weak service names:

```

DataService
HelperService
UtilityService
ManagerService

```

A good service answers: what business capability does this class own?

Stateless Service Design

Most Spring services should be stateless. A service should not store user-specific or request-specific data in fields.

Bad:

```

@Service
public class CheckoutService {

    private Long currentUserId;
}

```

```

public void setCurrentUserId(Long userId) {
    this.currentUserId = userId;
}

public void checkout() {
    // Uses currentUserId
}
}

```

Better:

```

@Service
public class CheckoutService {

    public Receipt checkout(Long userId, CheckoutRequest request) {
        // userId is passed into the method
    }
}

```

Use fields for dependencies, not changing request data.

Good fields:

```

private final OrderRepository orderRepository;
private final PaymentClient paymentClient;

```

Risky fields:

```

private Long currentUserId;
private List<Item> currentCart;
private BigDecimal temporaryTotal;

```

Transaction Boundaries

A transaction groups database operations so they succeed or fail together.

```

@Transactional
public void transfer(Long fromAccountId, Long toAccountId, BigDecimal amount) {
    Account from = accountRepository.findById(fromAccountId).orElseThrow();
    Account to = accountRepository.findById(toAccountId).orElseThrow();

    from.withdraw(amount);
    to.deposit(amount);
}

```

If withdrawing succeeds but depositing fails, the whole transaction should roll back. In Spring, `@Transactional` usually belongs on service methods because the service method represents a complete business operation.

Separation of Concerns

Each layer has a job.

Controller:

```
@PostMapping
public OrderResponse createOrder(@RequestBody CreateOrderRequest request) {
    return orderService.createOrder(request);
}
```

Service:

```
public OrderResponse createOrder(CreateOrderRequest request) {
    validateOrder(request);
    Order order = Order.createFrom(request);
    Order saved = orderRepository.save(order);
    return OrderResponse.from(saved);
}
```

Repository:

```
public interface OrderRepository extends JpaRepository<Order, Long> {
}
```

Entity:

```
public class Order {
    public void cancel() {
        if (status == OrderStatus.SHIPPED) {
            throw new OrderCannotBeCancelledException();
        }

        status = OrderStatus.CANCELLED;
    }
}
```

A helpful question:

Does this code care about HTTP, business rules, or persistence?

That usually tells you where it belongs.

Dependency Injection in Services

Prefer constructor injection:

```
@Service
public class CustomerService {
```

```

private final CustomerRepository customerRepository;
private final EmailService emailService;

public CustomerService(
    CustomerRepository customerRepository,
    EmailService emailService) {
    this.customerRepository = customerRepository;
    this.emailService = emailService;
}
}

```

Constructor injection is preferred because dependencies are clear, required dependencies cannot be skipped, testing is easier, and fields can be `final`.

Avoid field injection:

```

@Autowired
private CustomerRepository customerRepository;

```

Mental Checklist

When writing a service, ask:

1. Is the controller thin?
2. Are business rules in the service or domain model?
3. Are repositories only doing persistence?
4. Is request-specific data passed as method parameters?
5. Is the service stateless?
6. Is `@Transactional` placed around complete business operations?
7. Are dependencies injected through the constructor?
8. Can this service be unit tested without starting the whole web application?

Practice Exercises

Exercise 1: Move Logic Out of a Controller

Start with a controller that directly checks rules and calls a repository. Refactor it so:

- The controller only receives the request and returns the response.
- The service contains the business logic.
- The repository only handles data access.

Example scenario: `UserController` has a `deactivateUser()` endpoint.

Exercise 2: Build a Checkout Service

Create a `CheckoutService` with:

```

public CheckoutResponse checkoutBook(Long userId, Long bookId)

```

Rules:

- User must exist.
- Book must exist.
- User must not be blocked.
- Book must be available.
- Book should become unavailable after checkout.
- A checkout record should be saved.
- The method should use `@Transactional` .

Exercise 3: Identify Service Boundaries

Given these features:

- Register customer
- Update customer email
- Place order
- Cancel order
- Process payment
- Refund payment
- Reserve inventory
- Release inventory

Group them into services such as:

```
CustomerService
OrderService
PaymentService
InventoryService
```

Exercise 4: Stateless vs Stateful Service

Write a bad service like this:

```
@Service
public class CartService {
    private Long currentUserId;
    private List<Long> currentItemIds;
}
```

Then refactor it so request-specific data is passed into methods:

```
public CartResponse addItem(Long userId, Long itemId)
```

Exercise 5: Transaction Boundary Practice

Create a `BankTransferService` :

```
public void transfer(Long fromAccountId, Long toAccountId, BigDecimal amount)
```

Rules:

- Source account must exist.

- Destination account must exist.
- Amount must be positive.
- Source account must have enough balance.
- Source balance decreases.
- Destination balance increases.
- Both changes should succeed or fail together.

Exercise 6: Constructor Injection Practice

Create a service that depends on two repositories and one helper service.

```
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final ProductRepository productRepository;
    private final PricingService pricingService;

    public OrderService(
        OrderRepository orderRepository,
        ProductRepository productRepository,
        PricingService pricingService) {
        this.orderRepository = orderRepository;
        this.productRepository = productRepository;
        this.pricingService = pricingService;
    }
}
```

Exercise 7: Order Placement Workflow

Build:

```
public OrderResponse placeOrder(Long customerId, CreateOrderRequest request)
```

Rules:

- Customer must exist.
- Customer must be active.
- Each product must exist.
- Inventory must be sufficient.
- Total price must be calculated in the service or pricing component.
- Inventory must be reduced.
- Order must be saved.

Exercise 8: Service Method Naming

Rewrite vague method names into business-focused names.

Bad:

```
process()
handle()
```



```
doStuff()  
updateData()
```

Better:

```
placeOrder()  
cancelSubscription()  
approveLoanApplication()  
checkoutBook()  
transferFunds()
```

Exercise 9: Split an Overloaded Service

Create one large `ApplicationService` with methods for users, orders, payments, and inventory. Then split it into:

```
UserService  
OrderService  
PaymentService  
InventoryService
```

Exercise 10: Service Layer Unit Test Prep

Write a service method with dependencies injected through interfaces. Then write down what you would mock:

- Repository lookup
- Repository save
- Payment client
- Email sender
- Inventory service

Lab: Library Checkout API

Build a small Library Checkout API focused on service-layer design.

Goal

Create a Spring Boot service method that handles the full business workflow for checking out a book.

```
public CheckoutResponse checkoutBook(Long userId, Long bookId)
```

Business Rules

The checkout should succeed only when:

- The user exists.
- The book exists.
- The user is not blocked.
- The book is available.
- The book becomes unavailable after checkout.
- A checkout record is saved.
- The entire operation runs in one transaction.

Required Classes

```
User
Book
Checkout
UserRepository
BookRepository
CheckoutRepository
CheckoutService
CheckoutResponse
```

Optional:

```
UserNotFoundException
BookNotFoundException
BlockedUserException
BookUnavailableException
```

Entity Sketch

```
public class User {
    private Long id;
    private String name;
    private boolean blocked;

    public boolean isBlocked() {
        return blocked;
    }
}
```

```
public class Book {
    private Long id;
    private String title;
    private boolean available;

    public boolean isAvailable() {
        return available;
    }

    public void markCheckedOut() {
        this.available = false;
    }
}
```

```
public class Checkout {
    private Long id;
    private User user;
```

```

private Book book;
private LocalDateTime checkedOutAt;

public Checkout(User user, Book book) {
    this.user = user;
    this.book = book;
    this.checkedOutAt = LocalDateTime.now();
}
}

```

Service Task

Implement this:

```

@Service
public class CheckoutService {

    private final UserRepository userRepository;
    private final BookRepository bookRepository;
    private final CheckoutRepository checkoutRepository;

    public CheckoutService(
        UserRepository userRepository,
        BookRepository bookRepository,
        CheckoutRepository checkoutRepository) {
        this.userRepository = userRepository;
        this.bookRepository = bookRepository;
        this.checkoutRepository = checkoutRepository;
    }

    @Transactional
    public CheckoutResponse checkoutBook(Long userId, Long bookId) {
        // TODO:
        // 1. Find user
        // 2. Find book
        // 3. Reject blocked user
        // 4. Reject unavailable book
        // 5. Mark book checked out
        // 6. Save checkout
        // 7. Return response
    }
}

```

Expected Implementation Shape

```

@Transactional
public CheckoutResponse checkoutBook(Long userId, Long bookId) {
    User user = userRepository.findById(userId)
        .orElseThrow(() -> new UserNotFoundException(userId));
}

```

```

Book book = bookRepository.findById(bookId)
    .orElseThrow(() -> new BookNotFoundException(bookId));

if (user.isBlocked()) {
    throw new BlockedUserException(userId);
}

if (!book.isAvailable()) {
    throw new BookUnavailableException(bookId);
}

book.markCheckedOut();

Checkout checkout = new Checkout(user, book);
Checkout savedCheckout = checkoutRepository.save(checkout);

return CheckoutResponse.from(savedCheckout);
}

```

Controller Rule

The controller should stay thin:

```

@PostMapping("/users/{userId}/books/{bookId}/checkout")
public CheckoutResponse checkoutBook(
    @PathVariable Long userId,
    @PathVariable Long bookId) {
    return checkoutService.checkoutBook(userId, bookId);
}

```

No business logic should be inside the controller.

Lab Questions

1. Why does `@Transactional` belong on the service method?
2. Why should the controller not check whether the book is available?
3. Why is `CheckoutService` stateless?
4. What dependencies does `CheckoutService` need?
5. What unit tests would you write for this service?

Bonus Challenge

Add a rule:

A user cannot check out more than 3 books at a time.

Then update the service using:

```

int activeCheckouts = checkoutRepository.countByUserIdAndReturnedAtIsNull(userId);

```

If `activeCheckouts >= 3`, throw:

CheckoutLimitExceededException